

KONGU ENGINEERING COLLEGE
(AUTONOMOUS)

PERUNDURAI, ERODE – 638 060

EXPERIMENT - 8

PROJECT TITLE

**IOT BASED HVAC TEMPERATURE AND HUMIDITY
MONITORING SYSTEM**

SUBMITTED BY

ABISHEK R S	- 24MER002
GOWRISANKAR K	- 24MER010
NAVEENRAJ K	- 24MER035
PRAVEEN N M	- 24MER043

IOT BASED HVAC TEMPERATURE AND HUMIDITY MONITORING SYSTEM

1. Project review:

The IoT Based HVAC Temperature and Humidity Monitoring System is a compact prototype designed to continuously monitor the temperature and humidity of an indoor environment and provide suitable HVAC control. The system uses an ESP8266 NodeMCU as the central controller and a DHT11 sensor to measure temperature and humidity.

The DHT11 sensor continuously measures the environmental conditions and sends the readings to the ESP8266 NodeMCU. The controller processes these values and compares them with predefined temperature and humidity limits. When the measured conditions exceed the selected limits, the system activates suitable control devices through a relay module.

The ESP8266 has built-in Wi-Fi capability, allowing the system to be further developed for IoT-based remote monitoring, cloud data storage, mobile notifications, and real-time environmental monitoring.

2. Problem Statement:

HVAC systems are used to maintain comfortable temperature and humidity conditions in homes, offices, industries, hospitals, and other buildings. Manual monitoring of temperature and humidity can be inconvenient and may result in unnecessary energy consumption.

Traditional HVAC operation may not continuously respond to changes in environmental conditions. Excessive temperature or humidity can affect human comfort, equipment performance, and energy efficiency.

Therefore, there is a need for a simple and low-cost system that can continuously monitor temperature and humidity and automatically control suitable HVAC equipment when required.

The proposed system addresses these problems by integrating a DHT11 sensor, ESP8266 NodeMCU, relay module, buzzer, and LED. The sensor measures temperature and humidity, while the ESP8266 processes the readings and controls the HVAC system according to predefined conditions.

3. Proposed Solution:

The proposed solution consists of an ESP8266-based IoT HVAC Temperature and Humidity Monitoring System. The DHT11 sensor is used to continuously measure the temperature and humidity of the surrounding environment.

The ESP8266 processes the sensor readings and compares the measured values with predefined limits. If the temperature or humidity exceeds the selected limit, the relay module is activated to demonstrate suitable HVAC control. The LED provides a visual indication, and the buzzer can provide an audible warning during abnormal environmental conditions.

Proposed Operation:

1. Start the system and initialize the ESP8266.
2. Initialize the DHT11 sensor, relay, buzzer, and LED.
3. The DHT11 sensor measures temperature.
4. The DHT11 sensor measures humidity.
5. The sensor sends the measured data to the ESP8266.
6. ESP8266 processes the sensor readings.
7. The temperature and humidity values are compared with predefined limits.
8. If the conditions exceed the limits, the relay activates the HVAC control.
9. The LED and buzzer provide warning indications.
10. If the conditions are normal, the HVAC control remains OFF.
11. The system continuously repeats the monitoring process.

4. System Architecture:

The IoT Based HVAC Temperature and Humidity Monitoring System consists of five main sections:

Input Section:

DHT11 Sensor – Measures temperature and humidity.

The sensor sends the measured data to the ESP8266 NodeMCU.

Processing Section:

The ESP8266 NodeMCU acts as the central controller.

- Receives temperature and humidity data.
- Processes the sensor readings.
- Compares the values with predefined limits.
- Controls the relay, buzzer, and LED.

Actuation Section:

The Relay Module is used for HVAC control.

- The relay receives a control signal from the ESP8266.
- It can control a suitable fan, cooling system, or other HVAC load.

Indication Section:

The Buzzer and LED provide warning indications.

- Buzzer OFF and LED OFF → Temperature and humidity are normal.
- Buzzer ON and LED ON → Abnormal environmental condition detected.

Connectivity Section:

ESP8266 has built-in Wi-Fi capability.

The system can be further developed for:

- IoT-based HVAC monitoring.
- Cloud data storage.
- Mobile notifications.
- Remote environmental monitoring.
- Energy management.

5. Circuit Table:

Important Pin Mapping:

ESP8266 NodeMCU Pin Component

GPIO 2	D4 DHT11 Sensor → DATA
GPIO 14	D5 Relay Module → IN
GPIO 12	D6 Buzzer → Positive Terminal
GPIO 13	D7 LED → Anode through resistor
3.3V	– DHT11 Sensor → VCC
GND	– Common Ground

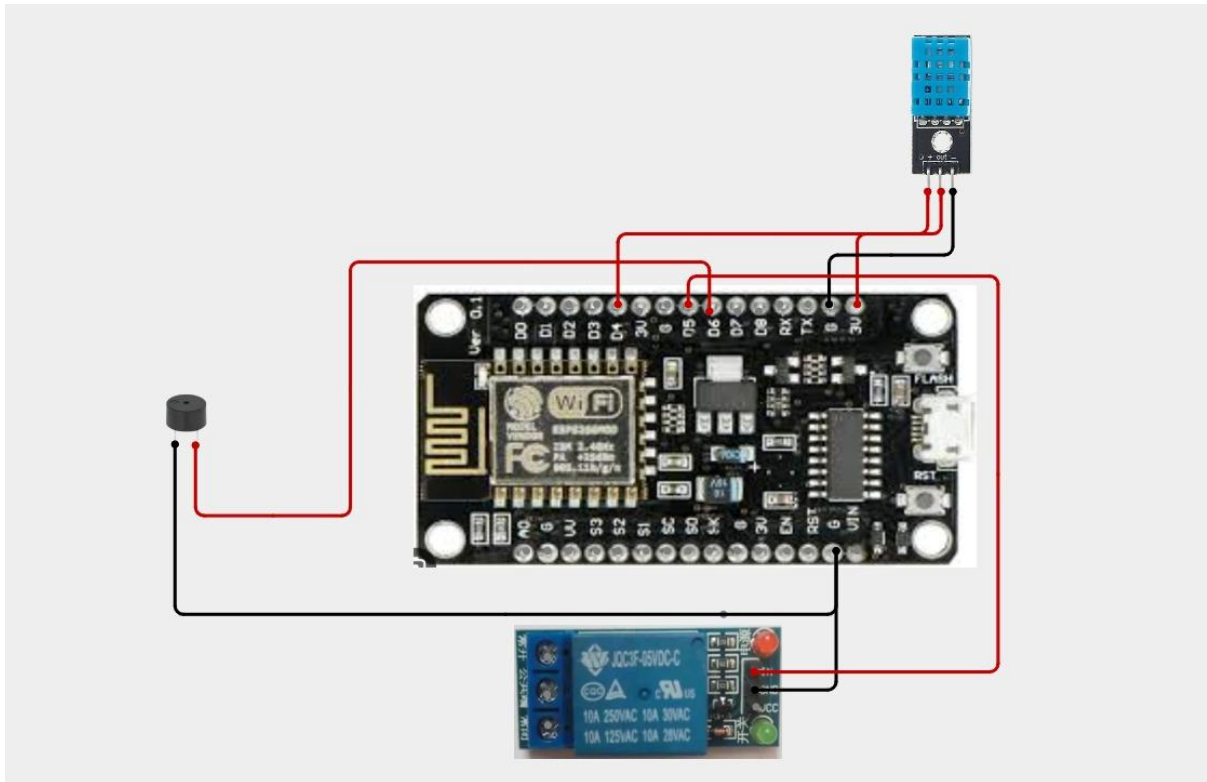
6. Circuit Design:

The circuit consists of an ESP8266 NodeMCU, DHT11 temperature and humidity sensor, relay module, buzzer, LED, breadboard, and jumper wires.

The DHT11 sensor DATA pin is connected to GPIO2 (D4) of the ESP8266. The sensor continuously measures the temperature and humidity of the surrounding environment.

The relay module is connected to GPIO14 (D5) and can be used to control a suitable HVAC device. The buzzer is connected to GPIO12 (D6) and provides an audible warning. The LED is connected to GPIO13 (D7) through a suitable resistor and provides a visual indication.

The ESP8266 continuously monitors the temperature and humidity readings. When the measured values exceed the predefined limits, the relay, LED, and buzzer are activated.



7. Algorithm:

1. Start the system.
2. Initialize the **ESP8266 NodeMCU, DHT11 sensor, relay, buzzer, and LED.**
3. Read the **temperature value** from the DHT11 sensor.
4. Read the **humidity value** from the DHT11 sensor.
5. Compare the temperature and humidity values with the predefined limits.
6. Check whether the **temperature or humidity exceeds the limit.**
7. If an abnormal condition is detected, turn **ON the relay, LED, and buzzer.**
8. Activate the **HVAC control system** and display **“HVAC CONTROL ACTIVE”**.
9. Otherwise, keep the **relay, LED, and buzzer OFF** and display **“ENVIRONMENT NORMAL”**.

8. Coding:

```
#include <ESP8266WiFi.h>
```

```
#include <ThingSpeak.h>
```

```
#include <DHT.h>
```

```
// =====
```

```
// IoT-BASED HVAC TEMPERATURE AND HUMIDITY MONITORING
```

```
// ESP8266 NodeMCU + DHT22 + Relay + Buzzer + WiFi
```

```
// + ThingSpeak
```

```
// =====
```

```
// =====
```

```
// WIFI SETTINGS
```

```
// =====
```

```
const char* WIFI_SSID = "praveen";
```

```
const char* WIFI_PASSWORD = "praveen12345";
```

```
// =====
```

```
// THINGSPEAK SETTINGS
```

```
// =====
```

```
// Your ThingSpeak Write API Key
```

```
const char* THINGSPEAK_API_KEY = "0IP4GOX0SMNK3JBV";
```

```
// CHANGE THIS TO YOUR ACTUAL CHANNEL ID
```

```
unsigned long CHANNEL_ID = 1234567;
```

```
// =====
```

```
// PIN DEFINITIONS
```

```
// =====
```

```
#define DHT_PIN 2    // D4 / GPIO2
```

```
#define DHT_TYPE DHT22
```

```
#define RELAY_PIN 14  // D5 / GPIO14
```

```
#define BUZZER_PIN 12  // D6 / GPIO12
```

```
// =====
```

```
// DHT OBJECT
```

```
// =====
```

```
DHT dht(DHT_PIN, DHT_TYPE);
```

```
// =====
```

```
// WIFI OBJECT
```

```
// =====
```

```
WiFiClient client;
```

```
// =====
```

```
// HVAC SETTINGS
```

```
// =====
```

```
// HVAC ON when temperature is above 28°C
```

```
float temperatureLimit = 28.0;
```

```
// Buzzer ON when temperature reaches 35°C
```

```
float dangerTemperature = 35.0;
```

```
// Most relay modules are ACTIVE LOW
```

```
bool relayActiveLow = true;
```

```
// =====
```

```
// THINGSPEAK TIMER
```

```
// =====
```

```
unsigned long lastUpdate = 0;
```

```
// ThingSpeak update interval
```

```
const unsigned long UPDATE_INTERVAL = 20000;
```

```
// =====
```

```
// SETUP
```

```
// =====
```

```
void setup()
```

```
{
```

```
  Serial.begin(9600);
```

```
  delay(1000);
```

```
  // Start DHT22
```

```
  dht.begin();
```

```
  // Relay
```

```
  pinMode(RELAY_PIN, OUTPUT);
```

```
  // Buzzer
```

```
  pinMode(BUZZER_PIN, OUTPUT);
```

```
// =====
```

```
// HVAC OFF AT STARTUP
```

```
// =====
```

```
if (relayActiveLow)
{
    digitalWrite(RELAY_PIN, HIGH);
}
else
{
    digitalWrite(RELAY_PIN, LOW);
}
```

```
// Buzzer OFF
```

```
digitalWrite(BUZZER_PIN, LOW);
```

```
// =====
```

```
// START MESSAGE
```

```
// =====
```

```
Serial.println();
```

```
Serial.println("=====");
```

```
Serial.println(" IoT HVAC MONITORING SYSTEM");
```

```
Serial.println("=====");
```

```
Serial.println("ESP8266 + DHT22 + Relay + Buzzer");
```

```
Serial.println("WiFi + ThingSpeak");
```

```
Serial.println("=====");
```

```
// =====  
// CONNECT TO WIFI  
// =====
```

```
WiFi.mode(WIFI_STA);
```

```
Serial.println();
```

```
Serial.print("Connecting to WiFi");
```

```
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
```

```
int attempts = 0;
```

```
while (WiFi.status() != WL_CONNECTED && attempts < 30)
```

```
{
```

```
    delay(500);
```

```
    Serial.print(".");
```

```
    attempts++;
```

```
}
```

```
Serial.println();
```

```
if (WiFi.status() == WL_CONNECTED)
{
  Serial.println("WiFi CONNECTED!");

  Serial.print("IP Address: ");

  Serial.println(WiFi.localIP());
}
else
{
  Serial.println("WiFi CONNECTION FAILED!");

  Serial.println("Check WiFi name and password.");
}
```

```
// =====
// START THINGSPEAK
// =====
```

```
ThingSpeak.begin(client);

Serial.println("ThingSpeak initialized.");

delay(2000);
```

```
}
```

```
// =====
```

```
// HVAC ON
```

```
// =====
```

```
void hvacON()
```

```
{
```

```
  if (relayActiveLow)
```

```
  {
```

```
    digitalWrite(RELAY_PIN, LOW);
```

```
  }
```

```
  else
```

```
  {
```

```
    digitalWrite(RELAY_PIN, HIGH);
```

```
  }
```

```
  Serial.println("HVAC STATUS : ON");
```

```
}
```

```
// =====
```

```
// HVAC OFF
```

```
// =====
```

```
void hvacOFF()
```

```
{  
  if (relayActiveLow)  
  {  
    digitalWrite(RELAY_PIN, HIGH);  
  }  
  else  
  {  
    digitalWrite(RELAY_PIN, LOW);  
  }  
  
  Serial.println("HVAC STATUS : OFF");  
}
```

```
// =====  
// BUZZER ON  
// =====
```

```
void buzzerON()  
{  
  digitalWrite(BUZZER_PIN, HIGH);  
  
  Serial.println("BUZZER STATUS : ON");  
}
```

```
// =====
```

```
// BUZZER OFF
```

```
// =====
```

```
void buzzerOFF()
```

```
{
```

```
    digitalWrite(BUZZER_PIN, LOW);
```

```
    Serial.println("BUZZER STATUS : OFF");
```

```
}
```

```
// =====
```

```
// MAIN LOOP
```

```
// =====
```

```
void loop()
```

```
{
```

```
// =====
```

```
// READ DHT22
```

```
// =====
```

```
float temperature = dht.readTemperature();
```

```
float humidity = dht.readHumidity();
```

```
Serial.println();
```

```
Serial.println("-----");
```

```
// =====
```

```
// CHECK FOR SENSOR ERROR
```

```
// =====
```

```
if (isnan(temperature) || isnan(humidity))
```

```
{
```

```
    Serial.println("DHT22 ERROR!");
```

```
    Serial.println("Check DHT22 wiring.");
```

```
    // Safety condition
```

```
    hvacOFF();
```

```
    buzzerOFF();
```

```
    delay(3000);
```

```
    return;
```

```
}
```

```
// =====  
// CHECK FOR IMPOSSIBLE VALUES  
// =====
```

```
if (temperature < -40.0 ||  
    temperature > 80.0 ||  
    humidity < 0.0 ||  
    humidity > 100.0)  
{
```

```
    Serial.println("INVALID DHT22 READING!");
```

```
    Serial.print("Temperature : ");
```

```
    Serial.print(temperature);
```

```
    Serial.println(" °C");
```

```
    Serial.print("Humidity  : ");
```

```
    Serial.print(humidity);
```

```
    Serial.println(" %");
```

```
Serial.println("Reading rejected for safety.");
```

```
// Do NOT operate HVAC
```

```
// Do NOT operate buzzer
```

```
hvacOFF();
```

```
buzzerOFF();
```

```
delay(3000);
```

```
return;
```

```
}
```

```
// =====
```

```
// DISPLAY VALID TEMPERATURE
```

```
// =====
```

```
Serial.print("Temperature : ");
```

```
Serial.print(temperature, 2);
```

```
Serial.println(" °C");
```

```
// =====  
// DISPLAY VALID HUMIDITY  
// =====
```

```
Serial.print("Humidity  :");
```

```
Serial.print(humidity, 2);
```

```
Serial.println(" %");
```

```
// =====  
// HVAC CONTROL  
// =====
```

```
int hvacStatus;
```

```
if (temperature > temperatureLimit)  
{
```

```
    Serial.println("Condition  : HIGH TEMPERATURE");
```

```
    hvacON();
```

```
    hvacStatus = 1;  
}
```

```
else  
{
```

```
    Serial.println("Condition : NORMAL");
```

```
    hvacOFF();
```

```
    hvacStatus = 0;  
}
```

```
// =====  
// BUZZER CONTROL  
// =====
```

```
int buzzerStatus;
```

```
if (temperature >= dangerTemperature)  
{
```

```
    buzzerON();
```

```
    buzzerStatus = 1;
```

```
}
```

```
else
```

```
{
```

```
    buzzerOFF();
```

```
    buzzerStatus = 0;
```

```
}
```

```
// =====
```

```
// SEND DATA TO THINGSPEAK
```

```
// =====
```

```
if (WiFi.status() == WL_CONNECTED &&
```

```
    millis() - lastUpdate >= UPDATE_INTERVAL)
```

```
{
```

```
    lastUpdate = millis();
```

```
    Serial.println();
```

```
    Serial.println("Sending data to ThingSpeak...");
```

```
// -----  
// FIELD 1 = TEMPERATURE  
// -----
```

```
ThingSpeak.setField(1, temperature);
```

```
// -----  
// FIELD 2 = HUMIDITY  
// -----
```

```
ThingSpeak.setField(2, humidity);
```

```
// -----  
// FIELD 3 = HVAC STATUS  
// -----
```

```
ThingSpeak.setField(3, hvacStatus);
```

```
// -----  
// FIELD 4 = BUZZER STATUS  
// -----
```

```
ThingSpeak.setField(4, buzzerStatus);
```

```
// -----  
// SEND DATA  
// -----  
  
int response = ThingSpeak.writeFields(  
    CHANNEL_ID,  
    THINGSPEAK_API_KEY  
);  
  
if (response == 200)  
{  
    Serial.println("ThingSpeak update SUCCESS!");  
}  
  
else  
{  
    Serial.print("ThingSpeak error code: ");  
  
    Serial.println(response);  
}  
}  
Serial.println("-----");  
  
delay(3000);  
}
```

Coding Output:

```
sketch_aug14b
DHT dht(DHT_PIN, DHT_TYPE);

// =====
// WIFI OBJECT
// =====

WiFiClient client;

// =====
// HVAC SETTINGS
// =====

// HVAC ON when temperature is above 28°C
float temperatureLimit = 28.0;

// Buzzer ON when temperature reaches 35°C
float dangerTemperature = 35.0;

// Most relay modules are ACTIVE LOW
bool relayActiveLow = true;

// =====
// THINGSPEAK TIMER
// =====

unsigned long lastUpdate = 0;

// ThingSpeak update interval
const unsigned long UPDATE_INTERVAL = 20000;
```

Done uploading.

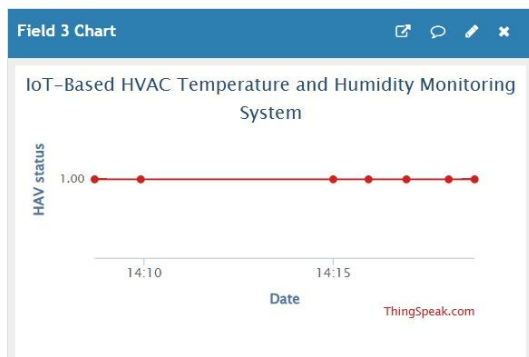
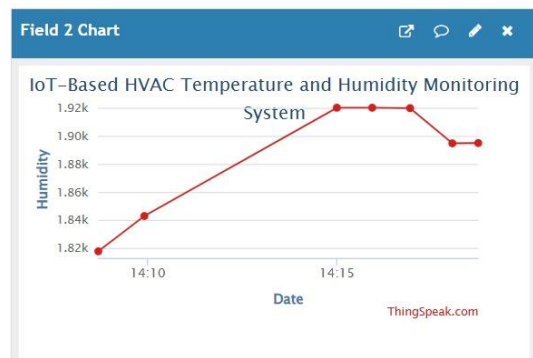
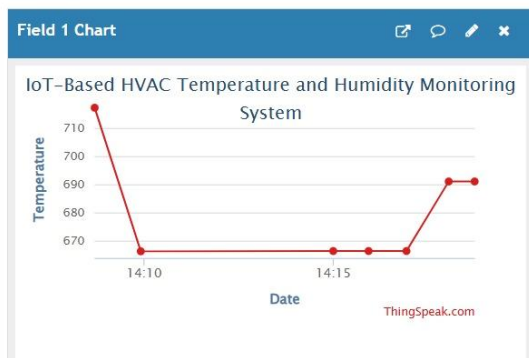
COM6

Send

DHT22 ERROR!
Check DHT22 wiring.
HVAC STATUS : OFF
BUZZER STATUS : OFF

DHT22 ERROR!
Check DHT22 wiring.
HVAC STATUS : OFF
BUZZER STATUS : OFF

☒ Autoscroll ☐ Show timestamp Newline 9600 baud Clear output



9. System Working:

Power ON the System:

The ESP8266 NodeMCU is powered ON.

Initialize Components:

The soil moisture sensor, relay module, buzzer, and LED are initialized.

Measure Temperature

The DHT11 sensor continuously measures the surrounding temperature.

Measure Humidity

The DHT11 sensor continuously measures the humidity level.

Send Sensor Data:

The DHT11 sends the temperature and humidity readings to the ESP8266.

Process the Data:

The ESP8266 receives and processes the sensor readings.

Check Environmental Conditions

The measured temperature and humidity are compared with predefined limits.

Abnormal Condition

If temperature or humidity exceeds the selected limit:

- Relay → ON
- HVAC Control → Activated
- LED → ON
- Buzzer → ON

Normal Condition

If temperature and humidity are within the normal range:

- Relay → OFF
- HVAC Control → OFF

- LED → OFF
- Buzzer → OFF

Repeat the Process:

- The system continuously measures temperature and humidity and automatically controls the HVAC system.

10. Prototype Implementation:

The prototype is implemented using an ESP8266 NodeMCU, DHT11 sensor, relay module, buzzer, LED, breadboard, and jumper wires.

The DHT11 sensor continuously measures the temperature and humidity of the surrounding environment. The ESP8266 receives the sensor readings and compares them with predefined limits.

Hardware Setup

Connect the DHT11 DATA pin → D4 (GPIO2) of ESP8266.

Connect the Relay IN pin → D5 (GPIO14).

Connect the Buzzer → D6 (GPIO12).

Connect the LED → D7 (GPIO13) through a suitable resistor.

Connect DHT11 VCC → 3.3V.

Connect all GND terminals to a common ground.

Connect the relay to a suitable HVAC demonstration load.

Upload the program to the ESP8266 using the Arduino IDE.

Power ON the prototype.

Observe the temperature and humidity readings.

Increase the temperature or humidity beyond the predefined limit.

When the conditions return to normal, the system automatically switches OFF the warning devices.

Prototype Working:

- The ESP8266 NodeMCU is powered ON and initializes all connected components.
- The DHT11 sensor measures temperature and humidity.
- The sensor sends the readings to the ESP8266.
- The ESP8266 processes the sensor data.
- The temperature and humidity values are compared with predefined limits.
- If either value exceeds the limit, the relay activates the HVAC control.
- The LED provides a visual indication.
- The buzzer provides an audible warning.
- When the environmental conditions return to normal, the relay and warning devices are switched OFF.
- The system continuously repeats the monitoring process.

11. Bill of Materials:

S.No.	Component	Specification	Quantity
1	ESP8266	NodeMCU Wi-Fi Development Board	1
2	DHT11 Sensor	Temperature and Humidity Sensor	1
3	Relay Module	1-Channel Relay	1
4	Buzzer	3.3V/5V Buzzer	1
5	LED	Standard Indicator LED	1
6	Resistor	Suitable LED Current Limiting Resistor	1
7	Breadboard	Standard Prototype Board	1
8	Jumper Wires	Male-Male / Male-Female	1 Set
9	USB Cable	Micro-USB	1
10	Power Supply	Suitable Regulated Supply	1

12. Results & Performance Analysis:

- The DHT11 sensor successfully measures the temperature and humidity of the environment.
- The ESP8266 NodeMCU successfully receives and processes the sensor readings.
- The system provides continuous monitoring of HVAC environmental conditions.
- When temperature or humidity exceeds the predefined limit, the relay is activated.
- The LED provides a visual warning indication.
- The buzzer provides an audible alert.
- When the environmental conditions return to the normal range, the HVAC control and warning devices are switched OFF.
- The system provides automatic and continuous monitoring of temperature and humidity

Results

The DHT11 sensor successfully measures temperature and humidity.

The ESP8266 NodeMCU successfully processes the sensor data.

The system continuously monitors environmental conditions.

High temperature or humidity is automatically detected.

The relay successfully demonstrates HVAC control.

The LED provides a visual warning indication.

The buzzer provides an audible alert.

The system automatically returns to normal operation when conditions become safe.

The prototype demonstrates the basic operation of an IoT Based HVAC Temperature and Humidity Monitoring System.

Performance Analysis:

Parameter	Expected Performance
Temperature Monitoring	Continuous
Humidity Monitoring	Continuous
Sensor Response	Real-time Reading
High Temperature Alert	Buzzer ON
High Humidity Alert	Buzzer ON
HVAC Control	Relay ON
Normal Condition	Relay, Buzzer and LED OFF